



UTM
UNIVERSITI TEKNOLOGI MALAYSIA

FACULTY OF COMPUTING

UNIVERSITI TEKNOLOGI MALAYSIA

OBJECT ORIENTED PROGRAMMING - SECJ2154

MINI PROJECT

PREPARED BY : GROUP

NAME	MATRIC NO
NURUL ADRIANA BINTI KAMAL JEFRI	A23CS0258
AHMAD ZIYAAD BIN MOHD ABBAS	A23CS0206

PREPARED FOR:

DR. MUHAMMAD KHATIBSYARBINI ABD RAHIM

Library Management System

– OOP Mini Project

This OOP mini project simulates a Library Management System that allows users to add, remove, and borrow books using core object-oriented programming concepts such as inheritance, polymorphism, composition, and exception handling.

Team Members

Name	Responsibilities
AHMAD ZIYAAD BIN MOHD ABBAS (A23CS0206)	Data structures, relationships, exception handling
NURUL ADRIANA BINTI KAMAL JEFRI (A23C0258)	Inheritance, polymorphism, documentation & testing

Features Implemented

- Add list, find, and remove books in a library
- Borrowed book tracking for members
- Custom exception for invalid or missing books
- Abstract superclass `User` with subclasses `Member` and `Librarian`
- Method overriding (`calculateFine()`, `displayRole()`)
- Dynamic binding with polymorphism

Object-Oriented Concepts with Implementation & Code Examples

Concept	Implementation Explanation	Code Example
Inheritance	Member and Librarian are subclasses that extend the abstract superclass User.	<pre>abstract class User { protected String name; protected String id; public abstract void displayRole(); public abstract double calculateFine(int daysLate); } class Member extends User { public void displayRole() { System.out.println("Role: Member"); } public double calculateFine(int daysLate) { return daysLate * 0.50; } } class Librarian extends User { public void displayRole() { System.out.println("Role: Librarian"); } public double calculateFine(int daysLate) { return 0.0; } }</pre>
Method Overriding	Subclasses override methods from User such as displayRole() and calculateFine().	<pre>@Override public void displayRole() { System.out.println("Role: Member"); } @Override public double calculateFine(int daysLate) { return daysLate * 0.50; }</pre>
Polymorphism	A User reference points to a Member or Librarian, allowing dynamic behavior.	<pre>User u1 = new Member("Adriana", "M002"); User u2 = new Librarian("Siti", "L001"); u1.displayRole(); // Calls Member's method u2.displayRole(); // Calls Librarian's method</pre>
Dynamic Binding	Java decides at runtime which version of the overridden method to call.	<pre>User user = new Member("Ali", "M003"); System.out.println("Fine: RM" + user.calculateFine(5));</pre>
Encapsulation	Class fields are private, and accessed through public methods.	<pre>class Book { private String title; public Book(String title) { this.title = title; } public String getTitle() { return title; } }</pre>
Composition	LibraryCard owns a Member instance. If the member is deleted, the card is affected.	<pre>class LibraryCard { private Member member; public LibraryCard(Member m) { this.member = m; } public void showCardDetails() { System.out.println("Library Card belongs to: " + member.name); } }</pre>
Association	Library can be associated with a Librarian (they can exist independently).	<pre>class Library { private Librarian librarian; public void assignLibrarian(Librarian l) { this.librarian = l; } }</pre>

Aggregation	Library uses a Librarian, but if Library is deleted, Librarian remains intact.	<pre>Library lib = new Library(); Librarian l = new Librarian("Daniel", "L003"); lib.assignLibrarian(l); lib = null; // Library deleted, Librarian still exists l.displayRole();</pre>
Exception Handling	Handles input validation and custom exceptions like BookNotFoundException using try-catch.	<pre>if (title.trim().isEmpty()) { throw new IllegalArgumentException("Title cannot be empty."); } try { lib.findBook("9999"); } catch (BookNotFoundException e) { System.out.println("Caught: " + e.getMessage()); }</pre>

Sample Output

```
=== Adding Books ===
Book added: The Hobbit
Book added: The Alchemist

=== Listing All Books ===
Current books in library:
[The Hobbit] by J.R.R. Tolkien (ISBN: 1111)
[The Alchemist] by Paulo Coelho (ISBN: 2222)

=== Finding Book ===
[The Hobbit] by J.R.R. Tolkien (ISBN: 1111)

=== Removing Book ===
Removed book with ISBN: 1111

=== Listing Remaining Books ===
Current books in library:
[The Alchemist] by Paulo Coelho (ISBN: 2222)

=== Member Borrows a Book ===
Ziyaad borrowed: The Alchemist
Ziyaad has borrowed:
[The Alchemist] by Paulo Coelho (ISBN: 2222)

=== Member Info ===
Role: Member
Fine for 3 late days: RM1.5

=== Polymorphism Test ===
Role: Member
Fine for 5 late days: RM2.5
```

```
Role: Librarian
Fine for 5 late days: RM0.0

=== Association Test ===
Librarian assigned to library: Farah
Library deleted (set to null). Librarian still exists:
Role: Librarian
Fine for 7 late days: RM0.0

=== Aggregation Test ===
Library 2 now has librarian: Daniel
Library 2 deleted (set to null). Librarian still exists and works:
Role: Librarian
Fine for 10 late days: RM0.0

=== Composition Test ===
Library Card belongs to: Hakim (ID: M003)
Member object deleted (set to null). Trying to use LibraryCard...
Library Card has no valid owner.
Warning: LibraryCard still exists, but is now detached from its original owner.

=== Exception Handling Test ===
Caught exception: Book not found: ISBN 9999
```

How To Run

1. Compile the program
2. Run the program

Link To README.md

1. Path on Github : StudentFile/Nurul Adriana/MiniProject
2. Link to Github :
<https://github.com/Dr-Khatib/OOP-Section3-2425-2/tree/main/StudentFile/Nurul%20Adriana/MiniProject>

UML Class Diagram

